DART

Standard Input / Output System

A Complete Beginner to Advanced Guide

stdin | stdout | stderr

Topics Covered:

Reading user input | Writing console output | Handling errors
Type conversions | CLI programs | Real-world patterns

dart:io Library | Exam & Interview Ready

Table of Contents

CHAPTER 1

stdin — Standard Input

> *stdin is the default channel through which a running program receives input from the user (or another process). In Dart, it is accessed via the dart:io library.*

1.1 What is stdin?

When you run a Dart program in the terminal, the operating system provides three default communication channels:

- stdin — data coming INTO the program (keyboard input)
- stdout — data going OUT of the program (normal output)
- stderr — error messages going OUT of the program

stdin stands for Standard Input. It is a stream — a continuous flow of bytes that your program can read. By default, it is connected to the keyboard, but it can also be piped from a file or another program.

In Dart, stdin is a static property of the IOSink class, accessible through the dart:io library:

```
import 'dart:io';

void main() {
  // Access the global stdin stream
  stdout.write('Enter your name: ');
  String? name = stdin.readLineSync();
  print('Hello, $name!');
}
```

1.2 Why is stdin Used?

stdin is used whenever your program needs to:

- Ask the user for data during runtime (name, age, choice, etc.)
- Build interactive command-line tools (calculators, menus, quizzes)
- Process piped input from other programs or files
- Accept configuration values without hard-coding them

Without stdin, programs would need all data at compile time — making them inflexible. stdin enables dynamic, interactive programs.

1.3 How stdin.readLineSync() Works

Step-by-step breakdown:

- **stdin** — the global standard input stream object
- **.readLineSync()** — a synchronous method that BLOCKS execution until the user presses Enter

What happens internally when readLineSync() is called:

1. The program PAUSES and waits for keyboard input
2. The user types characters and presses Enter
3. The method reads all characters up to (but not including) the newline
4. It returns the collected characters as a String? value
5. Program execution RESUMES after the return

The 'Sync' in readLineSync means SYNCHRONOUS — the program waits (blocks) until input is received. This contrasts with asynchronous input which would use callbacks or streams.

```
import 'dart:io';

void main() {
  stdout.write('Enter your age: ');  // prompt without newline
  String? input = stdin.readLineSync();  // BLOCKS here
  // Execution continues only after user presses Enter
  print('You entered: $input');
}
```

1.4 Why readLineSync() Returns String?

The return type is String? (nullable String) — not String. This is an important distinction in Dart's null-safety system.

It returns null when:

- The input stream reaches End-Of-File (EOF)
- stdin is redirected from a file that has been fully read
- The user sends EOF signal (Ctrl+D on Linux/Mac, Ctrl+Z on Windows)
- The stream is closed before input is received

Since these situations can legitimately occur, Dart forces you to handle the nullable case. If readLineSync() returned a non-nullable String and the stream ended, it would crash.

> *Best Practice: Always use the null-coalescing operator ?? to provide a fallback value, or use the null-aware operator ?. when working with the result.*

```
import 'dart:io';

void main() {
  stdout.write('Enter your name: ');

  // Option 1: null-coalescing (provide default)
  String name = stdin.readLineSync() ?? 'Unknown';

  // Option 2: explicit null check
  String? rawInput = stdin.readLineSync();
  if (rawInput == null) {
    print('No input received (EOF)');
    return;
  }
  print('Hello, $rawInput!');
}
```

1.5 Converting Input to int / double

readLineSync() always returns text (String). To use the input as a number, you must parse it. Dart provides built-in parse methods with error handling:

Converting to int:

```
import 'dart:io';

void main() {
  stdout.write('Enter your age: ');
  String? input = stdin.readLineSync();

  // Method 1: int.parse() — throws FormatException if invalid
  int age = int.parse(input!);

  // Method 2: int.tryParse() — returns null if invalid (safer)
  int? safeAge = int.tryParse(input ?? '');
  if (safeAge == null) {
    stderr.writeln('Error: Please enter a valid integer!');
    return;
  }

  print('Next year you will be ${safeAge + 1}');
}
```

Converting to double:

```
import 'dart:io';

void main() {
  stdout.write('Enter a decimal number: ');
  String? input = stdin.readLineSync();

  // double.tryParse() — handles both 3 and 3.14
  double? value = double.tryParse(input ?? '');
  if (value == null) {
    stderr.writeln('Invalid number format!');
    return;
  }

  print('Square: ${value * value}');
}
```

parse() vs tryParse() — important distinction:

- **int.parse('abc')** — throws FormatException (use in try-catch)
- **int.tryParse('abc')** — returns null (preferred for user input)

1.6 stdin Code Examples

Example 1: Greeting Program

```
import 'dart:io';

void main() {
  // Ask for first name
  stdout.write('Enter your first name: ');
  String firstName = stdin.readLineSync() ?? 'Friend';

  // Ask for last name
  stdout.write('Enter your last name: ');
  String lastName = stdin.readLineSync() ?? '';

  print(''); // blank line
  print('Welcome, $firstName $lastName!');
  print('Nice to meet you.');
}

// Sample Output:
// Enter your first name: Ali
// Enter your last name: Khan
//
// Welcome, Ali Khan!
// Nice to meet you.
```

Example 2: Number Input with Validation

```
import 'dart:io';

void main() {
  int? number;

  // Keep asking until valid input received
  while (number == null) {
    stdout.write('Enter a whole number: ');
    String? input = stdin.readLineSync();
    number = int.tryParse(input ?? '');

    if (number == null) {
      stderr.writeln('Invalid input. Please enter digits only.');
    }
  }

  print('You entered: $number');
  print('Doubled: ${number * 2}');
  print('Squared: ${number * number}');
}
```

Example 3: Simple CLI Calculator using stdin

```
import 'dart:io';

void main() {
  print('=== Dart CLI Calculator ===');
  print('');

  // Read first number
  stdout.write('Enter first number: ');
  double? num1 = double.tryParse(stdin.readLineSync() ?? '');

  // Read operator
  stdout.write('Enter operator (+, -, *, /): ');
  String op = stdin.readLineSync() ?? '';

  // Read second number
  stdout.write('Enter second number: ');
  double? num2 = double.tryParse(stdin.readLineSync() ?? '');

  // Validate inputs
  if (num1 == null || num2 == null) {
    stderr.writeln('Error: Invalid number input!');
    return;
  }

  // Calculate result
  double? result;
  switch (op) {
```

```
    case '+': result = num1 + num2; break;
    case '-': result = num1 - num2; break;
    case '*': result = num1 * num2; break;
    case '/':
      if (num2 == 0) {
        stderr.writeln('Error: Division by zero!');
        return;
      }
      result = num1 / num2;
      break;
    default:
      stderr.writeln('Error: Unknown operator "$op"');
      return;
  }

  print('');
  print('Result: $num1 $op $num2 = $result');
}

// Sample Output:
// === Dart CLI Calculator ===
// Enter first number: 15
// Enter operator (+, -, *, /): *
// Enter second number: 4
// Result: 15.0 * 4.0 = 60.0
```

CHAPTER 2

stdout — Standard Output

> *stdout is the default output channel for a program. It is connected to the terminal by default, displaying everything your program wants to show to the user.*

2.1 What is stdout?

stdout stands for Standard Output. In Dart, it is an IOSink object from dart:io that writes data to the terminal. Every time you call print() in Dart, it internally uses stdout.

stdout is a stream — your program pushes data into it, and the terminal displays it. stdout is line-buffered by default (output is flushed when a newline is written).

```
import 'dart:io';

void main() {
  // These two lines do the SAME thing:
  print('Hello World');           // uses stdout internally
  stdout.writeln('Hello World');  // explicit stdout
}
```

2.2 print() vs stdout.write()

This is one of the most important distinctions to understand for CLI development:

Feature	print()	stdout.write()
Newline added?	YES — always adds \n at end	NO — you control newlines
Usage	print('Hello')	stdout.write('Hello')
Output type	Calls toString() automatically	Requires String argument
Flushing	Auto-flushes (newline triggers)	May need manual flush
Best for	Debug output, quick printing	Prompts, progress bars, formatting
Multiple prints	Each on new line	All on same line until \n written

Visual comparison:

```
import 'dart:io';

void main() {
  // print() — each call adds a newline
  print('One');   // prints: One\n
  print('Two');   // prints: Two\n
  // Terminal shows:
  // One
  // Two

  print('--- separator ---');

  // stdout.write() — NO automatic newline
  stdout.write('One');  // prints: One  (cursor stays here)
  stdout.write('Two');  // prints: Two  (cursor stays here)
  stdout.write('\n');  // NOW move to next line
  // Terminal shows:
  // OneTwo
}
```

Why use stdout.write() for prompts?

```
import 'dart:io';

void main() {
  // WRONG — cursor goes to new line, input appears on next line
  print('Enter name: ');
  // Terminal: Enter name:
  //           <cursor here — looks odd>

  // CORRECT — cursor stays on same line as prompt
  stdout.write('Enter name: ');
  // Terminal: Enter name: <cursor here>
  String? name = stdin.readLineSync();
}
```

2.3 stdout.writeln() Usage

stdout.writeln() is a convenience method that writes text AND adds a newline — exactly like print(). The key differences:

- **print()** — implicitly calls toString() on the argument, handles null gracefully
- **stdout.writeln()** — part of the IOSink API, more control, works with encoding

```
import 'dart:io';

void main() {
  // All three produce identical terminal output:
```

```
  print('Hello, World!');
  stdout.writeln('Hello, World!');
  stdout.write('Hello, World!\n');

  // stdout.writeln with empty line:
  stdout.writeln();  // just a blank line
  print('');         // same thing
}
```

2.4 When to Use stdout in Real CLI Tools

In simple scripts, print() is fine. For real CLI tools, stdout gives you precision:

- **Progress indicators:** stdout.write('Processing...') then update the line
- **Same-line prompts:** stdout.write('Password: ') before reading input
- **Structured output:** Build formatted tables without unwanted line breaks
- **Encoding control:** stdout.encoding lets you set UTF-8 or other encodings
- **Piping output:** stdout can be redirected to files: dart app.dart > output.txt

```
import 'dart:io';

void main() {
  // Inline progress simulation
  stdout.write('Loading');
  for (int i = 0; i < 5; i++) {
    stdout.write('.');  // dots appear on same line
    // In real app: sleep(Duration(milliseconds: 200))
  }
  stdout.writeln(' Done!');
  // Output: Loading..... Done!

  // Table formatting
  print('+----------+-------+');
  print('| Name     | Score |');
  print('+----------+-------+');
  stdout.write('| Ali      | ');
  stdout.write('  95   |');
  stdout.writeln();
  print('+----------+-------+');
}
```

2.5 stdout Code Examples

Example 1: Formatted Student Report

```dart
import 'dart:io';

void printDivider() => print('=' * 40);

void printRow(String label, String value) {
  stdout.write('  $label');
  // Pad to align values
  int spaces = 20 - label.length;
  stdout.write(' ' * spaces);
  stdout.writeln(': $value');
}

void main() {
  printDivider();
  print('       STUDENT REPORT CARD');
  printDivider();
  printRow('Name', 'Ahmed Ali');
  printRow('Roll No', '2024-CS-101');
  printRow('Mathematics', '92/100');
  printRow('Physics', '88/100');
  printRow('Dart Programming', '97/100');
  printDivider();
  printRow('Total', '277/300');
  printRow('Grade', 'A+');
  printDivider();
}
```

Example 2: stdout with Encoding

```dart
import 'dart:io';
import 'dart:convert';

void main() {
  // Set encoding explicitly (important for special characters)
  stdout.encoding = utf8;

  // Now safely print Unicode / special characters
  print('Dart Version: 3.x');
  print('Status: Running OK');
  print('Check: All systems go');
}
```

CHAPTER 3

stderr — Standard Error

> *stderr is the standard error output stream. It is intentionally separate from stdout so that error messages and normal output can be routed independently.*

3.1 What is stderr?

stderr stands for Standard Error. Like stdout, it writes to the terminal by default — but it is a completely separate stream. This separation is fundamental to Unix/Linux design philosophy.

In Dart, stderr is accessed via dart:io just like stdout:

```
import 'dart:io';

void main() {
  stdout.writeln('This is normal output');  // goes to stdout
  stderr.writeln('This is an error message'); // goes to stderr
}

// In the terminal, both appear the same visually,
// but they are different streams underneath.
```

3.2 Why stderr is Separate from stdout

The separation enables powerful shell operations:

- **Redirect only output:** dart app.dart > output.txt (errors still show on screen)
- **Redirect only errors:** dart app.dart 2> errors.log (normal output still on screen)
- **Redirect both:** dart app.dart > out.txt 2> err.txt (complete separation)
- **Suppress errors:** dart app.dart 2>/dev/null (hide all errors)

> *Real-world example: A data processing script might produce thousands of lines of output. If errors were mixed in, finding them would be impossible. With stderr, errors are always captured separately.*

Three key reasons to use stderr:

6. Clean output: Normal output can be piped without error noise
7. Logging: Errors can be logged to separate files for monitoring
8. Tooling: IDEs, CI systems, and debuggers parse stderr separately

3.3 Real Use Cases for stderr

- **File not found:** When a required file is missing, report via stderr
- **Invalid arguments:** CLI tools report bad arguments to stderr
- **Network failures:** Connection errors go to stderr, not mixed with data
- **Validation errors:** Input validation failures belong in stderr
- **Debug tracing:** In non-debug builds, trace logs go to stderr
- **Fatal errors:** Any error that terminates the program

The convention: if your program exits with a non-zero exit code (error), use stderr. If it exits with 0 (success), use stdout.

3.4 stderr Code Examples

Example 1: Basic stderr Usage

```
import 'dart:io';

void main() {
  // Normal program output -> stdout
  stdout.writeln('Program started successfully.');
  stdout.writeln('Processing data...');

  // Simulating an error condition
  bool errorOccurred = true;

  if (errorOccurred) {
    stderr.writeln('[ERROR] Data processing failed!');
    stderr.writeln('[ERROR] Check input file format.');
    exit(1);  // Non-zero exit code signals failure
  }

  stdout.writeln('Done!');
  exit(0);  // Zero exit code signals success
}
```

Example 2: File Validation with stderr

```
import 'dart:io';

void processFile(String filePath) {
  File file = File(filePath);

  if (!file.existsSync()) {
    stderr.writeln('[ERROR] File not found: $filePath');
```

```
    exit(1);
  }

  // File exists — proceed with processing
  stdout.writeln('Reading file: $filePath');
  String content = file.readAsStringSync();
  stdout.writeln('Lines: ${content.split("\n").length}');
}

void main(List<String> args) {
  if (args.isEmpty) {
    stderr.writeln('Usage: dart app.dart <filename>');
    exit(1);
  }
  processFile(args[0]);
}
```

Example 3: Error Logging Pattern

```
import 'dart:io';

// Structured error logging helper
void logError(String message, {String? context}) {
  String timestamp = DateTime.now().toIso8601String();
  stderr.writeln('[$timestamp] ERROR: $message');
  if (context != null) {
    stderr.writeln('  Context: $context');
  }
}

void logInfo(String message) {
  stdout.writeln('[INFO] $message');
}

void main() {
  logInfo('Application starting...');

  stdout.write('Enter divisor: ');
  String? input = stdin.readLineSync();
  int? divisor = int.tryParse(input ?? '');

  if (divisor == null) {
    logError('Invalid input', context: 'Expected integer, got: $input');
    exit(1);
  }

  if (divisor == 0) {
    logError('Division by zero', context: 'Divisor cannot be zero');
    exit(1);
  }

  logInfo('100 / $divisor = ${100 / divisor}');
```

```
}
```

CHAPTER 4

Comparison Tables

4.1 print() vs stdout.write()

Aspect	print()	stdout.write()
Newline	Always adds \n	No newline added
Argument type	Any Object (calls toString())	String only
Null handling	Prints 'null' safely	Throws on null
Flush behavior	Auto-flushes on newline	May buffer; call flush()
Return value	void	void
Library needed	No import needed	import 'dart:io'
Use case	Debugging, quick output	CLI prompts, formatting
Same-line output	Not possible	Yes

4.2 stdin vs Other Input Methods

Method	stdin.readLineSync()	Command-line Args	Environment Vars
When available	During runtime	At startup only	Before program runs
Interactive?	Yes	No	No
Return type	String?	List<String>	String?
How to access	stdin.readLineSync()	main(List<String> args)	Platform.environment
Use case	Interactive tools	Scripts, automation	Config, secrets
Blocking?	Yes (waits for Enter)	No	No

4.3 stdout vs stderr

Aspect	stdout	stderr
Purpose	Normal program output	Error and diagnostic output
Dart object	stdout (IOSink)	stderr (IOSink)
File descriptor	1	2
Default destination	Terminal	Terminal
Shell redirect	> file.txt	2> errors.txt
Pipe behavior	Can be piped | to other programs	Not piped by default
Buffering	Line buffered	Unbuffered (immediate)
Used for	Results, reports, data	Warnings, errors, diagnostics
Exit code relation	Used with exit(0)	Used with exit(1) or non-zero

CHAPTER 5

Real-World Usage Patterns

5.1 Complete Interactive CLI Application

This example combines stdin, stdout, and stderr in a realistic menu-driven program:

```
import 'dart:io';

// --- Helper functions ---

String prompt(String message) {
  stdout.write(message);
  return stdin.readLineSync() ?? '';
}

int? promptInt(String message) {
  String input = prompt(message);
  return int.tryParse(input);
}

void showError(String msg) => stderr.writeln('[ERROR] $msg');
void showInfo(String msg)  => stdout.writeln('[INFO]  $msg');

void printMenu() {
  print('');
  print('============================');
  print('     STUDENT GRADE TOOL     ');
  print('============================');
  print(' 1. Add student grade');
  print(' 2. View all grades');
  print(' 3. Calculate average');
  print(' 4. Exit');
  print('============================');
}

// --- Main program ---

void main() {
  List<Map<String, dynamic>> students = [];

  while (true) {
    printMenu();
    int? choice = promptInt('Select option (1-4): ');

    if (choice == null) {
      showError('Please enter a number between 1 and 4.');
      continue;
```

```
    }

    switch (choice) {
      case 1:
        String name = prompt('Student name: ');
        int? grade = promptInt('Grade (0-100): ');
        if (grade == null || grade < 0 || grade > 100) {
          showError('Grade must be between 0 and 100.');
          break;
        }
        students.add({'name': name, 'grade': grade});
        showInfo('Added: $name with grade $grade');
        break;

      case 2:
        if (students.isEmpty) {
          showInfo('No students added yet.');
          break;
        }
        print('');
        print('Name            | Grade');
        print('----------------+------');
        for (var s in students) {
          String name = s['name'].toString().padRight(16);
          print('$name| ${s["grade"]}');
        }
        break;

      case 3:
        if (students.isEmpty) {
          showError('No students to average.');
          break;
        }
        double avg = students.fold(0, (s, e) => s + e['grade']) / students.length;
        showInfo('Class average: ${avg.toStringAsFixed(1)}');
        break;

      case 4:
        showInfo('Exiting. Goodbye!');
        exit(0);

      default:
        showError('Invalid choice. Enter 1-4.');
    }
  }
}
```

This program demonstrates the correct pattern: stdout for menus and data, stderr for errors, and stdin for all user interaction — a professional CLI design.

5.2 Reading Multiple Inputs in a Loop

```
import 'dart:io';

void main() {
  print('Enter numbers one per line.');
  print('Type "done" to finish and see the sum.\n');

  List<double> numbers = [];

  while (true) {
    stdout.write('Number: ');
    String? input = stdin.readLineSync();

    if (input == null || input.toLowerCase() == 'done') {
      break;
    }

    double? num = double.tryParse(input);
    if (num == null) {
      stderr.writeln('Skipping invalid input: "$input"');
      continue;
    }
    numbers.add(num);
  }

  if (numbers.isEmpty) {
    stdout.writeln('No valid numbers entered.');
    return;
  }

  double sum = numbers.reduce((a, b) => a + b);
  double avg = sum / numbers.length;

  print('\n--- Results ---');
  print('Count  : ${numbers.length}');
  print('Sum    : $sum');
  print('Average: ${avg.toStringAsFixed(2)}');
}
```

CHAPTER 6

Summary & Quick Reference

6.1 Core Concepts Summary

Stream	Object	Key Method(s)	Returns	Use For
stdin	Stdin (dart:io)	readLineSync()	String?	Reading user input
stdout	IOSink (dart:io)	write(), writeln()	void	Normal program output
stderr	IOSink (dart:io)	write(), writeln()	void	Errors and diagnostics
print()	Built-in function	print(value)	void	Quick/debug output

6.2 Key Takeaways

- **stdin.readLineSync()** always returns String? — always handle the null case
- **int.tryParse() / double.tryParse()** are safer than parse() for user input
- **stdout.write()** does NOT add newline — use for prompts and inline output
- **stdout.writeln()** is equivalent to print() but requires dart:io import
- **stderr** is for errors only — never mix errors into stdout
- **exit(0)** = success, exit(1) or non-zero = failure — always use correctly

6.3 Common Mistakes to Avoid

Mistake	Problem	Correct Approach
Using print() for prompts	Cursor moves to new line	Use stdout.write() for prompts
Not handling null from readLineSync()	Null pointer risk	Use ?? operator or null check
Using int.parse() without try-catch	Crashes on bad input	Use int.tryParse() instead
Writing errors to stdout	Breaks piping & logging	Always use stderr for errors
Forgetting import 'dart:io'	stdin/stdout undefined	Add import at top of file

Mistake	Problem	Correct Approach
exit(1) on success	Wrong exit code semantics	exit(0) = success, exit(1) = error

6.4 Quick Reference Cheat Sheet

```
// ============================================================
// DART I/O QUICK REFERENCE
// ============================================================
import 'dart:io';

// --- stdin: Read User Input ---
String?  raw    = stdin.readLineSync();           // nullable
String   safe   = stdin.readLineSync() ?? '';    // with default
int?     intVal = int.tryParse(raw ?? '');        // to int
double?  dblVal = double.tryParse(raw ?? '');     // to double

// --- stdout: Normal Output ---
print('Hello');                  // auto newline
stdout.write('Enter: ');         // NO newline (for prompts)
stdout.writeln('Hello');         // WITH newline
stdout.write('Line 1\nLine 2'); // manual newlines

// --- stderr: Error Output ---
stderr.writeln('Error: something went wrong');
stderr.write('[ERROR] $message\n');

// --- Exit Codes ---
exit(0);   // Success
exit(1);   // General error
exit(2);   // Misuse / bad arguments

// --- Prompt Pattern (Best Practice) ---
stdout.write('Your name: ');
String name = stdin.readLineSync() ?? 'Unknown';
// ============================================================
```

End of Document — Dart Standard I/O System

Generated for comprehensive study — stdin | stdout | stderr